Alissa Tsay

16-bit CPU in Logisim

Architecture Design Report

Contents

- **Part 1: Basic CPU Implementation**
    - **Overview**
    - **8-bit ALU Implementation**
    - **16x8 Bit Register File**
    - **PC Unit**
    - **CLU as a RAM Look Up table**
    - **CLU as a RAM Look Up table**
    - **Full 16-bit CPU with CLU as a RAM component**
- **Part 2: Advanced CPU**
    - **Implementing support for Branch instructions**
        - **CPU Structure**
        - **Implementing ALU with a zero flag and AND gate**
        - **Implementing adder and multiplexer**
        - **Implementing an OR gate**
    - **Building support for Jump instructions**
        - **Redesigning PC**
        - **Full CPU with jump support**

Part 1: Basic CPU

Overview

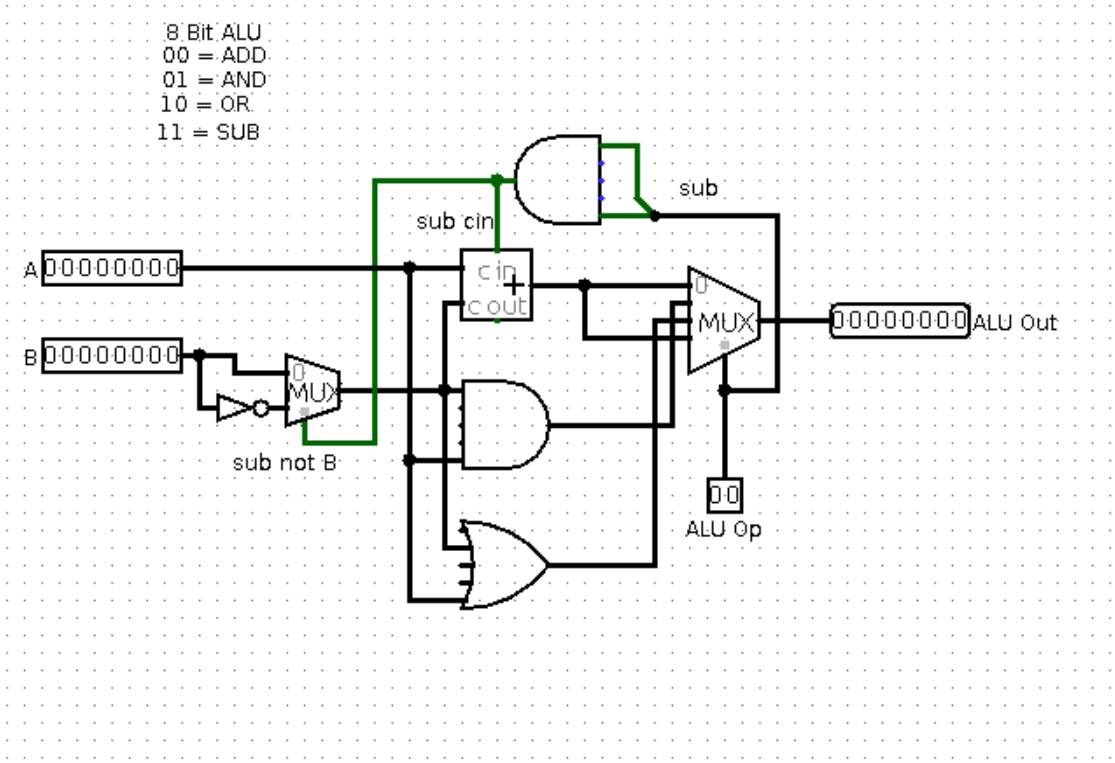
The CPU I have built has an 8-bit data width register file, with 16 registers, RAM based lookup-table Control Logic Unit, 8-bit data width and 8-bit address-width RAM.

The instruction word is 16 bits, distributed as follows:

I/R-type <1bit> opcode <3bits> rs <4bits> rt <4bits> rd <4bits>

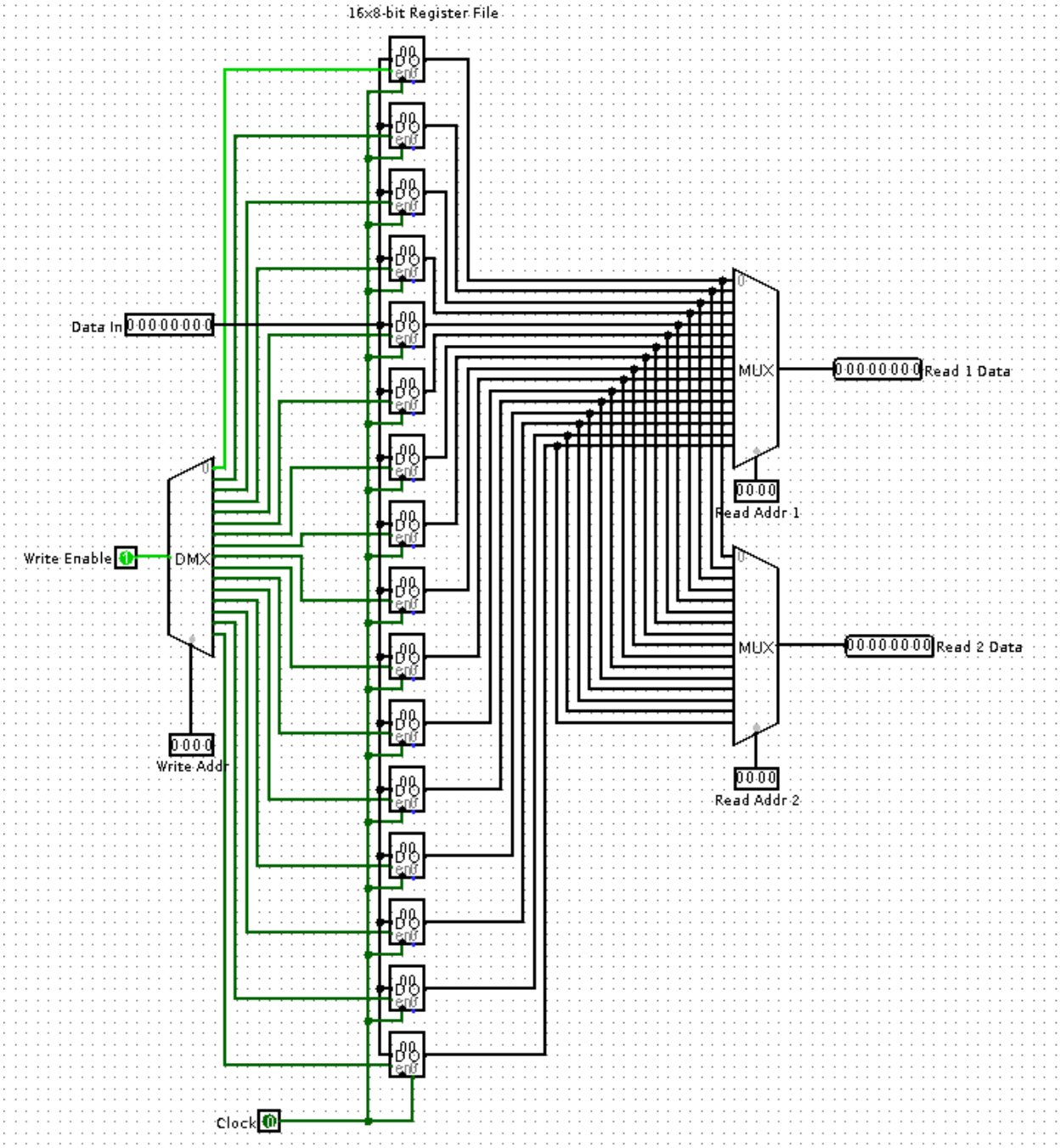
It has the following individual parts:

8-bit ALU



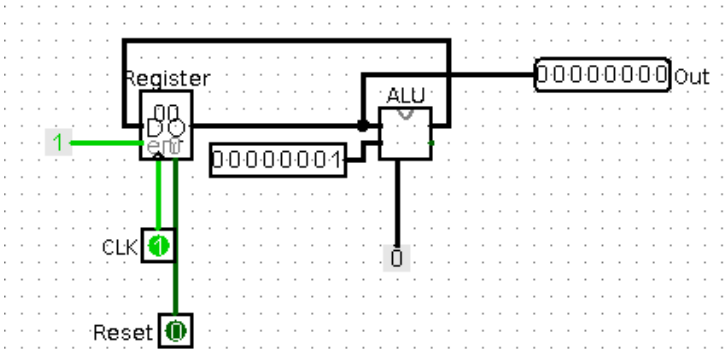
A simple 8-bit ALU, which can perform add (00), and (01), or (10), and sub (11) operations.

16x8 Bit Register File



An 8-bit data width register file with 16 registers, numbered $0 through $15. The Write Address, Read Address 1 and 2 are 4 bits each.

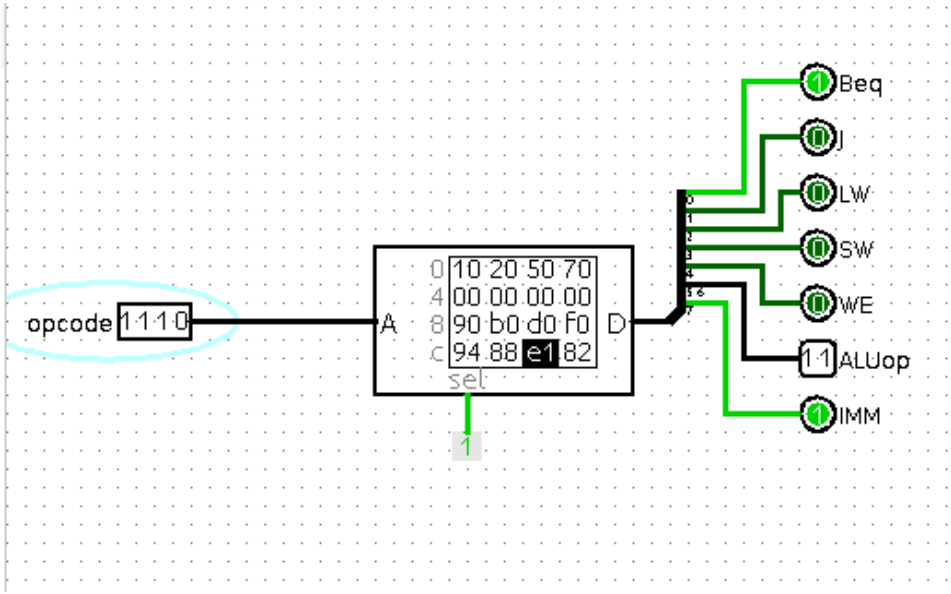
PC Unit



The PC unit also closely follows my implementation from lab 4, with the only difference being the data bit width change from 4 bits to 8. The next address for the basic CPU is calculated within the unit by adding 1 to the previous PC address stored in the register. This implementation was later changed to support absolute jump instructions.

CLU as a RAM Look Up table

The CLU is implemented as a RAM-based lookup table. I used Logisim ROM component to implement it. It had an address bit width of 4 bits, corresponding to the length of the R/T type bit + opcode (3 bits) in the instruction word; and 8 bit width data fields.



The 8 output bits are distributed as follows:

op	R/I + opcode	IMM	ALUop	WE	SW	LW	J	Beq	Hex
add	0000	0	00	1	0	0	0	0	10
and	0001	0	01	1	0	0	0	0	20
or	0010	0	10	1	0	0	0	0	50
sub	0011	0	11	1	0	0	0	0	70
nop	0100	0	00	0	0	0	0	0	00
nop	0101	0	00	0	0	0	0	0	00
nop	0110	0	00	0	0	0	0	0	00
nop	0111	0	00	0	0	0	0	0	00
addi	1000	1	00	1	0	0	0	0	90
andi	1001	1	01	1	0	0	0	0	b0
ori	1010	1	10	1	0	0	0	0	d0
subi	1011	1	11	1	0	0	0	0	f0
lw	1100	1	00	1	0	1	0	0	94
sw	1101	1	00	0	1	0	0	0	88
beq	1110	0	11	0	0	0	0	1	61
j	1111	1	00	0	0	0	1	0	82

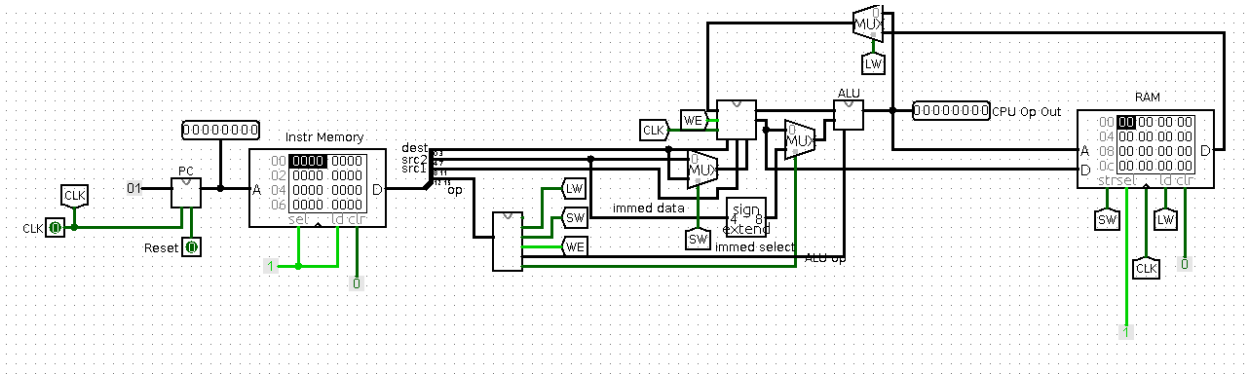
For this implementation, I have considered several ways to distribute the opcode fields. I have initially considered adding a separate funct field, as is done in the MIPS implementation. However, in my case, I have 8 immediate instructions which I could fit into 3 bits, giving me one extra bit for the immediate, however, with the addition of 0000 opcode to indicate an R-type instruction, I would now need 4 bits for the opcode, which is the same number I got by keeping both R-type and I-type instructions in the opcode. I have then considered using separate bits for LW and SW instructions, however, such implementation would also add unnecessary bits into

the opcode. In the end, I have decided to keep a separate R/I type bit, and distribute the values of the other 3 bits between all supported instructions.

The values to load can be found in the CLU_opcode.hex file included in the submission.

Basic 16-bit CPU with CLU as a RAM component

Structure



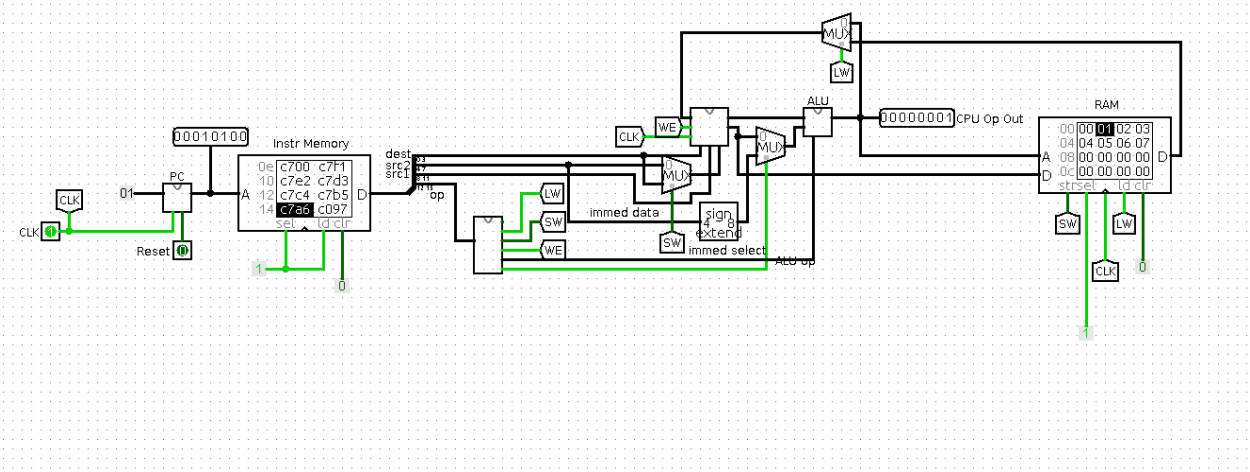
In my implementation I attempted to implement the order of registers in the instruction word after the MIPS Processor standard. The following is the distribution of fields in my instruction word, left to right, from most significant to least significant bits:

I/R-type <1bit> opcode <3bits> rs <4bits> rt/imm <4bits> rd <4bits>

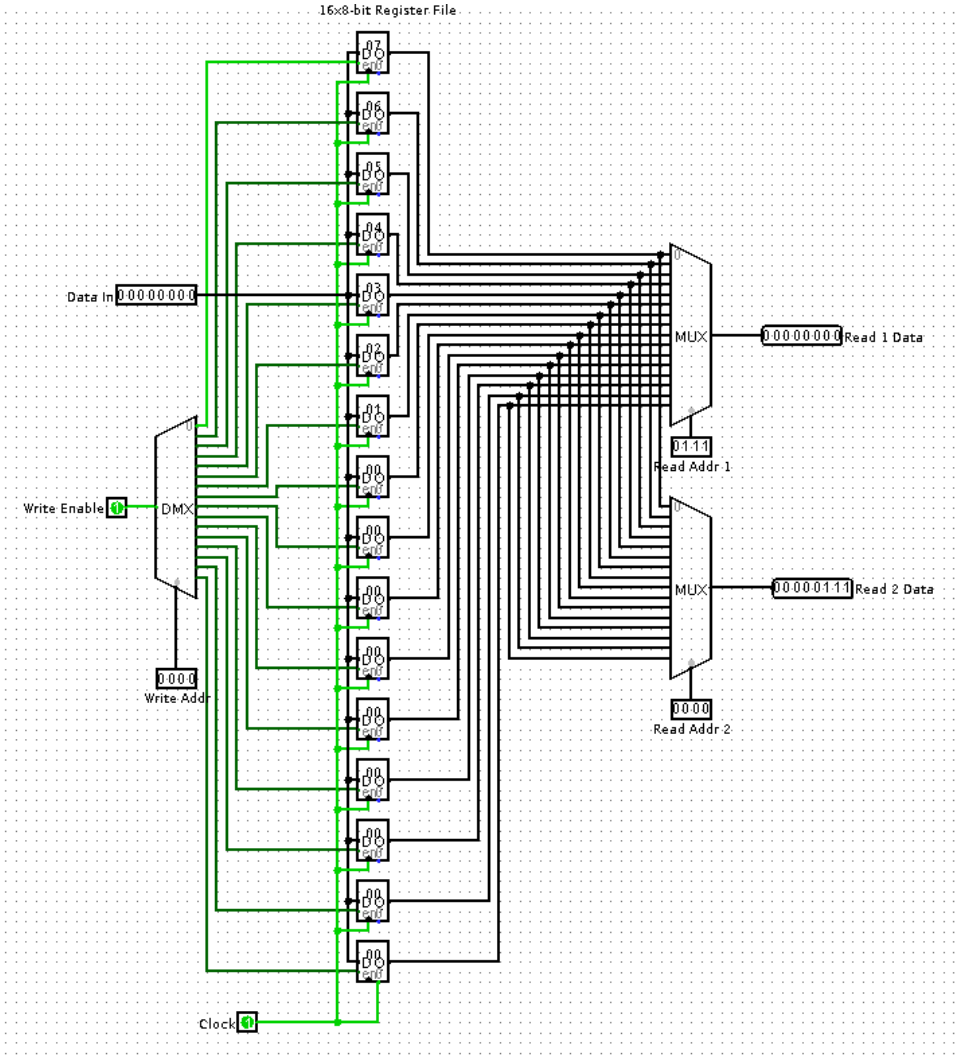
The assembly commands accepted by my assembler and the corresponding instruction word structure are as follows:

Assembly line	**Instruction Word**
add/and/or/sub $rd, $rs, $rt	R-/I + opcode (4 bits) | $rs (4) | $rt (4) | $rd (4)
addi/andi/ori/subi $rd, $rs, imm	R/I + opcode | $rs | imm | $rd
lw/sw $rd, imm($rs)	R/I + opcode | $rs | imm | $rd

After executing LW/SW custom program



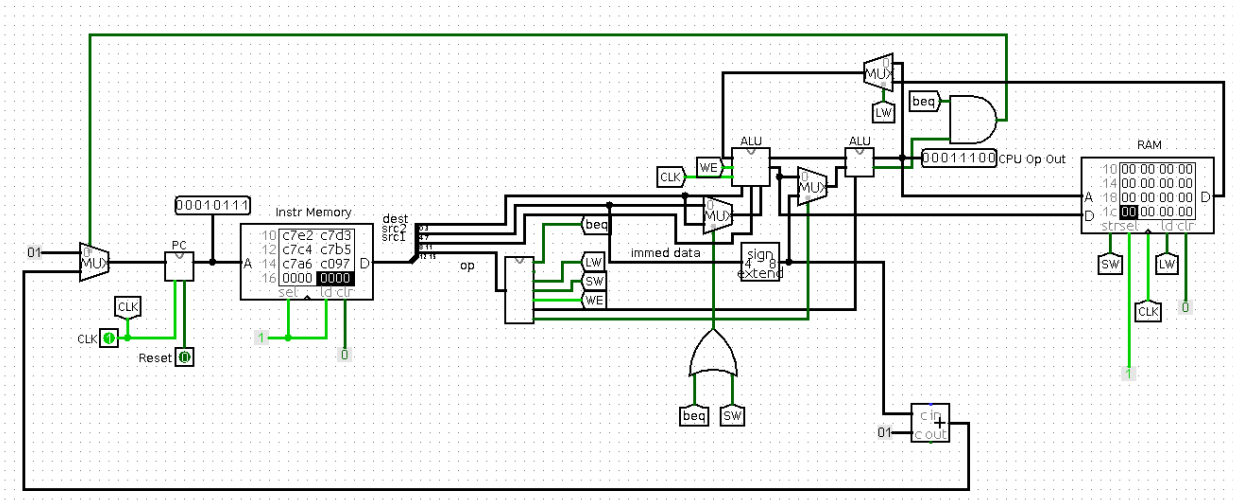
Reg File after executing the example:



Part 2: Advanced CPU

Implementing support for Branch instructions

CPU Structure:

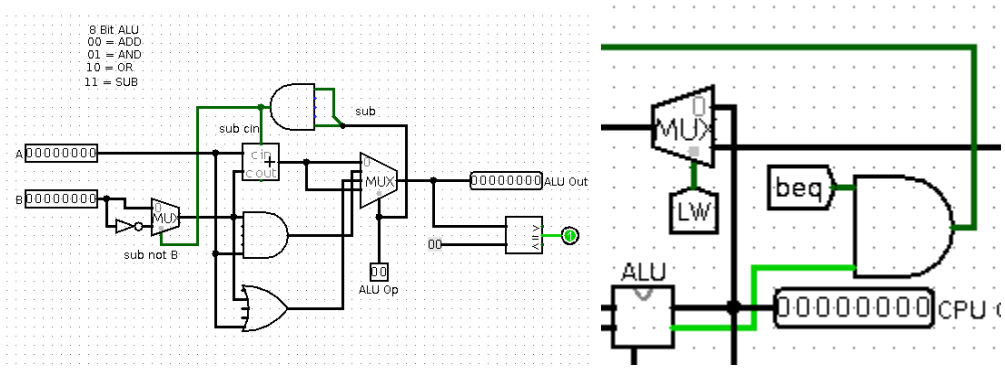


The assembly command and the corresponding instruction word structure are as follows:

Assembly line	Instruction Word
beq $rs, $rt, imm	R/I + opcode | $rs | imm | $rt

To adjust my CPU to support beq instructions, I had to implement the following:

ALU with a zero flag and AND gate

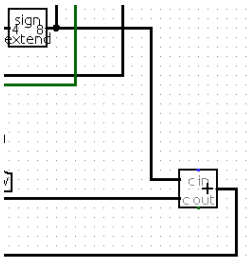


Since beq instructions perform the subtraction on the two values from the register file, and the branch is taken only when they are equal => the result of the subtraction is 0, I have added a

second output called zero flag into my ALU. I did so by using a Logisim comparator component with 8 bit inputs - one is the result of the ALU operation, the other is an 8-bit zero constant - and 1 bit output. The zero flag output sets to 1 if the two values are equal, and to 0 otherwise.

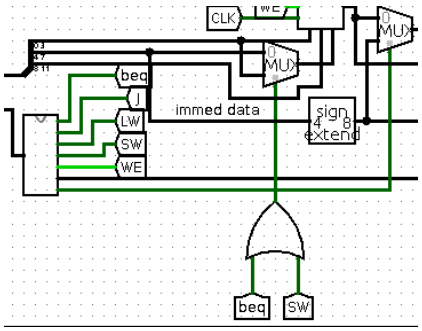
The AND gate then takes that value as an input, and a beq bit from the instruction word as the second input. The result of the AND operation is used as a selector into the MUX later on. This is necessary to not trigger the branch wherever the result of the ALU operation is 0, but no branch instruction has been issued.

An Adder and a Multiplexer



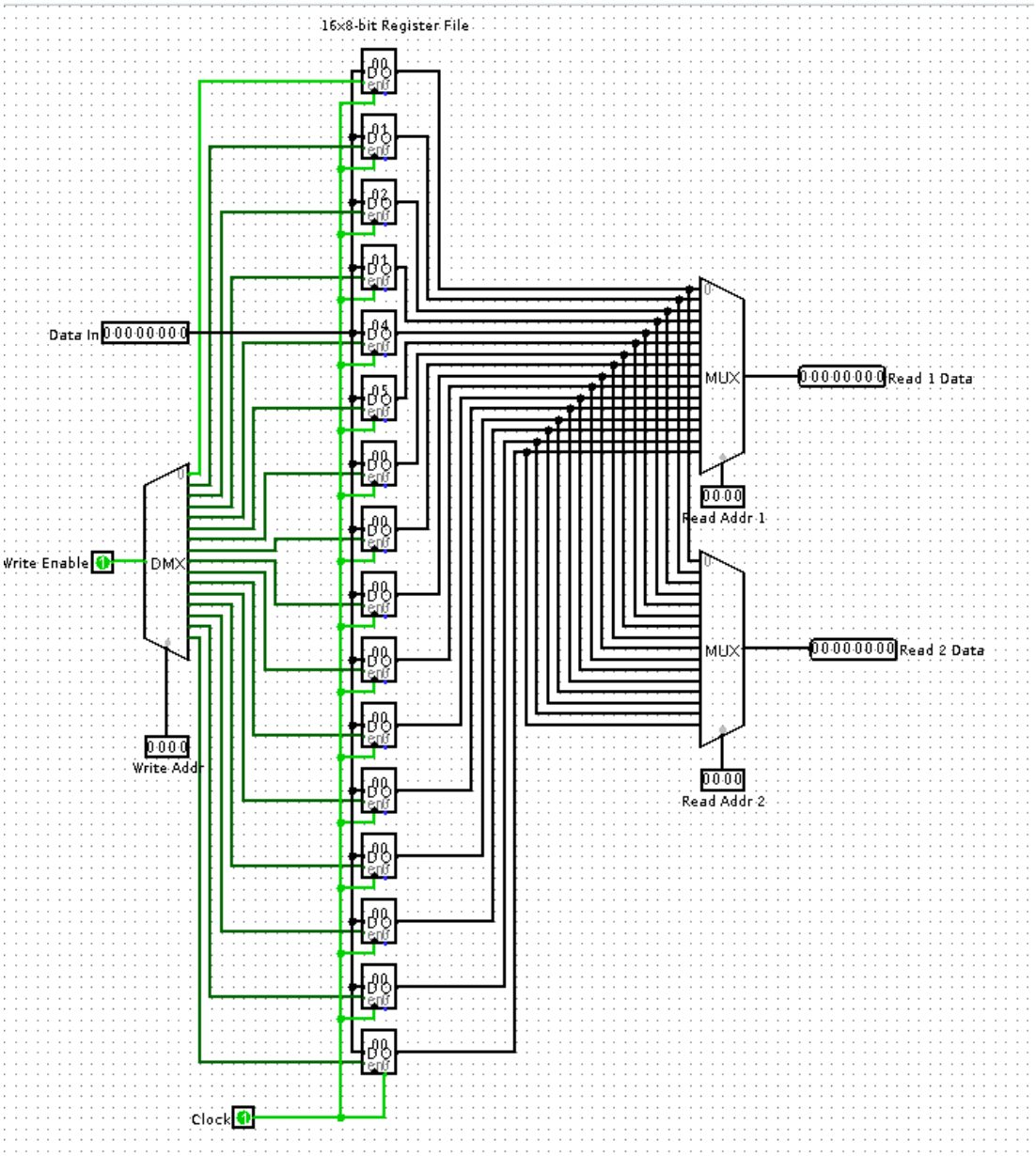
If the branch is taken the resulting address is calculated by adding the number of instructions to branch over + 1. My implementation for this part uses an adder component to add 1 to the branch number of instructions, and then uses a multiplexer to select between another hardwired 1 constant and the result of the previous addition. The zero flag AND beq is used as a selector, where 1 means that the branch is taken, and 0 means the opposite. The output value of the multiplexer is then wired into the PC, where the PC component adds it to the current address.

An OR gate



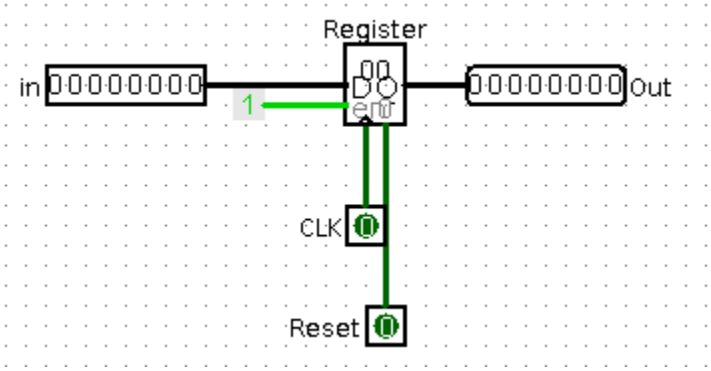
There was one last modification I had to make to insure the correct support of branch instructions. In the basic implementation, the only case when the $rd field had to be the second Read Address was the SW command. The SW thus, was used as a selector into the MUX with $rt and $rd as the two inputs. Since the branch instruction is structured in a similar way, I had to add an OR gate, feeding the beq and SW bits as inputs, and wiring the output into the MUX. Now, the destination is used as a second read address wherever either beq or SW are true.

Register File after executing the beq test example



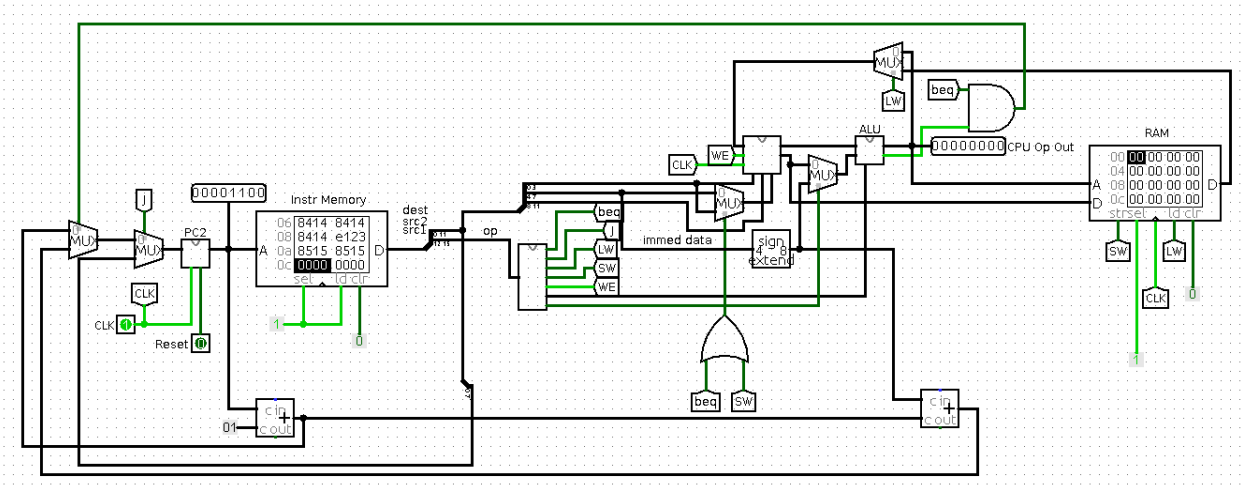
Building support for Jump instructions

Redesigned PC



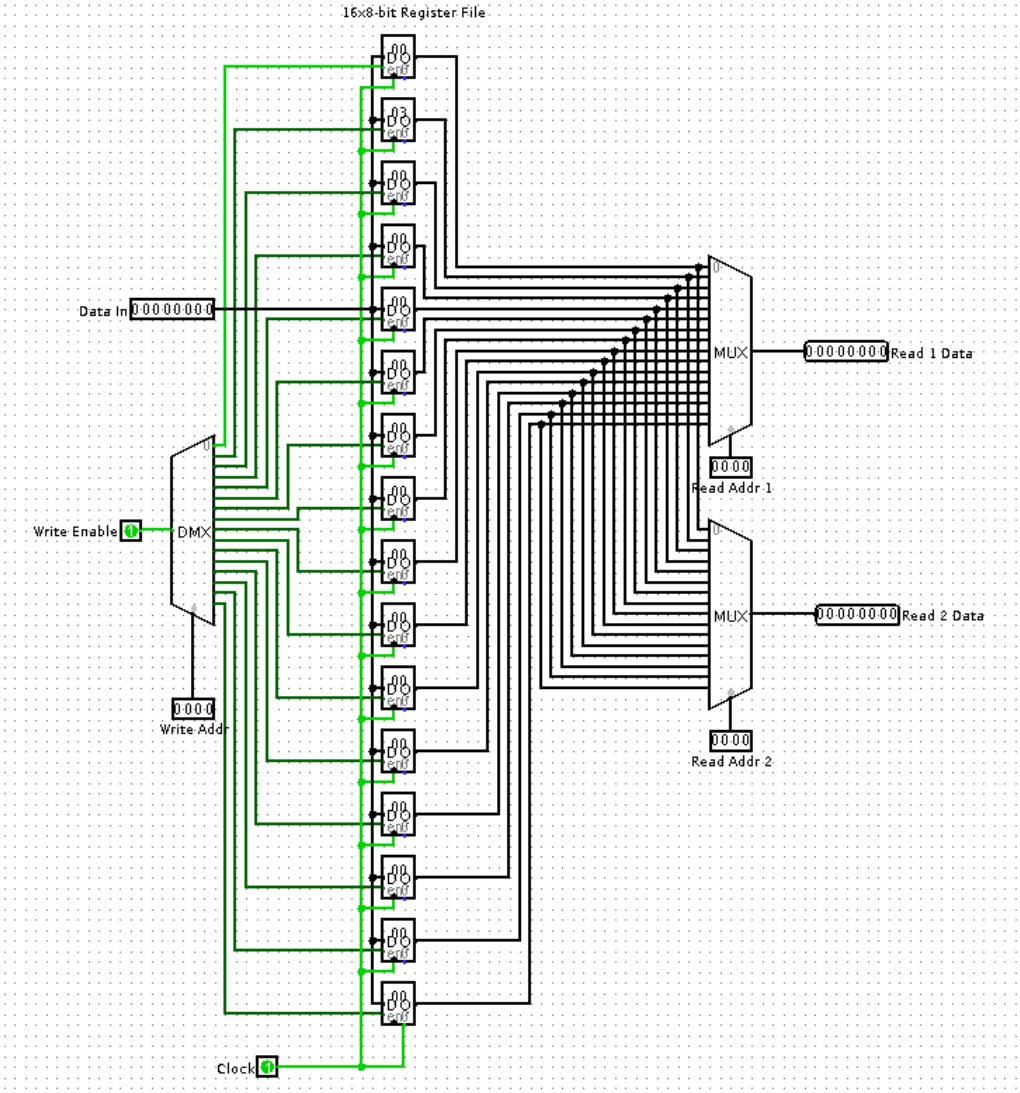
As was mentioned previously, my initial PC implementation would not be able to support an absolute address required for jump instructions. Thus, I had to change my implementation to simply include one register, the value to be written into it, the output, as well as the clock, reset, and enable pins. All logic related to address calculated was taken out into the CPU.

CPU with jump support



The assembly command and the corresponding instruction word structure are as follows:

Assembly line	Instruction Word
j label/imm	R/I + opcode | absolute address

To implement the support for jump instructions, I had to add one more adder, and one more multiplexer into my design. The first adder calculates the address of the next instruction when there are no jumps and no branches to be taken. The adder that previously calculated the number of instructions to jump over, now calculates the absolute address after adding the necessary number of instructions to skip to the PC address + 1 it receives as an input from the first adder. The second multiplexer receives either PC + 1 or the PC + 1 + branch as one of its inputs, and the absolute address from the jump instruction as the other. The jump bit from the instruction word is used as a selector, where 1 corresponds to jump, and 0 to no jump.

Register File after executing the jump example